JOHANNES KEPLER
UNIVERSITY LINZ

FIRSTNAME LASTNAME
Institute for Machine
Learning

@ firstname.lastname@students.jku.

July 29, 2026

Single-Task GRPO on MolecularIQ

Technical Report

How far does one task get you, and does it transfer?

Abstract We train Qwen2.5-0.5B-Instruct with Group Relative Policy Optimization (GRPO) separately on each of the three MolecularIQ task families – counting, indexing and constraint generation – and evaluate every resulting model on every task to separate specialization from transfer. Single-task training reliably improves the task it was trained on, but the gains do not transfer: cross-task cells are at or below the untrained baseline. Comparing our generated held-out sets against the official benchmark also exposes a flaw in our own constraint data, which a constant-answer control confirms. We report all results with confidence intervals and paired significance tests.

Keywords reinforcement learning, GRPO, molecular reasoning, LLM evaluation

Practical work, supervised by SUPERVISOR NAME. All code, configurations and evaluation results are available in the accompanying repository; `scripts/reproduce.sh` regenerates every number and figure in this report.

**JOHANNES KEPLER
UNIVERSITY LINZ**
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

1. Introduction

Large language models are increasingly asked to reason about molecules: to count functional groups, locate atoms by index, or generate a structure that satisfies a set of numeric constraints. Unlike open-ended chemistry questions, these tasks have a decisive property – the answer can be checked mechanically. MolecularIQ [7] exploits exactly this: it pairs generated questions over SMILES strings [10] with a symbolic verifier built on RDKit [4], so a model's answer is scored as objectively correct or incorrect rather than judged for plausibility.

A verifiable reward is precisely what reinforcement learning needs. Group Relative Policy Optimization (GRPO) [8] samples a group of completions per prompt, scores each with the verifier, and uses the within-group advantage as its learning signal, which removes the need for a separate value network. This makes it practical to fine-tune a small model on a single consumer-class GPU allocation, and it is the setting this work explores.

The obvious next step from such a setup is multitask training. Before that is worth attempting, however, a more basic question needs an answer: *how far does training on one task alone actually get you, and does any of it transfer?* If a model trained to count also improves at indexing, the tasks share structure that multitask training could amplify. If instead each model only improves at its own task – or actively degrades elsewhere – then multitask training is solving an interference problem, not harvesting a synergy, and should be designed accordingly.

This report answers that question empirically. We train four single-task GRPO models (counting, indexing, constraint generation, and a single-construct counting variant restricted to aromatic rings), then evaluate every model on every task family, forming a full model × task matrix. The diagonal of that matrix measures specialization; the off-diagonal measures transfer. We deliberately evaluate against two independent sources – our own held-out generated questions and the official MolecularIQ benchmark splits – because agreement between them is evidence that a result is real, and disagreement is itself diagnostic.

That second point turned out to matter more than expected. The two sources agree closely for counting and indexing but diverge by roughly a factor of nine for constraint generation, which led us to a concrete defect in our own dataset generator rather than in the models. We treat that finding as a result in its own right: it is a worked example of why a benchmark result needs a control, and Section 4.3 documents both the diagnosis and its cause.

The contributions of this report are: (i) a full single-task specialization-versus-transfer matrix for MolecularIQ under GRPO; (ii) a cross-validation of our generated evaluation data against the official benchmark; (iii) the diagnosis of a reward-hackable flaw in generated constraint questions, with a constant-answer control that quantifies it; and (iv) an evaluation harness that reports confidence intervals, paired significance tests and answer-diversity statistics by default.

2. Background and Setup

2.1 Tasks

MolecularIQ groups questions into three families that we treat as separate training tasks:

- **Counting** – how many instances of a construct does a molecule contain (rings, aromatic rings, carbon atoms, hydrogen-bond donors, ...)? The answer is an integer.
- **Indexing** – at which atom indices does a construct occur? The answer is a list of integers.
- **Constraint generation** – emit a SMILES string satisfying a set of property constraints (e.g. "at least two rings and fewer than six rotatable bonds"). The answer is a molecule, and the verifier checks it against the constraints.

The first two are analytic: the molecule is given and the model must read it. The third is generative and has many valid answers, which – as Section 4.3 shows – makes the difficulty of a question depend entirely on how tightly the constraints were drawn.

2.2 Model and training algorithm

All runs fine-tune Qwen2.5-0.5B-Instruct [6] in full (no adapters), using the GRPO implementation in TRL [9] with vLLM for rollout generation [3]. The reward is the official MolecularIQ verifier score (binary) with weight 1.0, plus a small format-shaping term with weight 0.2 that rewards a parseable answer block; the shaping term exists only to stop early training from being starved of signal by malformed output. We use 16 generations per prompt, a constant learning rate of 1×10^{-6} after warm-up, and $\beta = 0$ (no KL penalty, hence no reference model resident in memory). Full hyperparameters are in `configs/*.yaml` in the repository.

3. Method

3.1 Data

Training and held-out questions are generated with `scripts/create_datasets.py`, which draws molecules from the benchmark pool and emits questions in the official schema. Each task gets 20,000 training and 500 held-out validation questions, generated with seed 42. Questions are deduplicated, and for counting and indexing the fraction of questions whose answer is zero or empty is capped at 25% (`--max-zero-frac`) so that a policy cannot reward-hack by always answering "0". As we discuss in Section 4.3, this cap is *not* applied to constraint generation, which is the root of the defect we found.

3.2 Evaluation protocol

Every model is evaluated on every task family, against two sources:

Held-out generated sets the 500 unseen questions per task from our own generator, decoded greedily ($T = 0$), scored as pass@1. Because these come from the same generator as the training data, they measure whether GRPO *fit the distribution it was trained on* – an optimization diagnostic.

Official benchmark splits the corresponding `ml-jku/molecularIQ-v0.0` splits, sampled at $T = 1.0$ with $n = 3$ and scored as pass@3 [1], following the protocol of the benchmark paper. This is the generalization measure and the headline result.

Keeping both is deliberate. Selecting checkpoints on the official split would bias the number we report; we therefore use the held-out sets for any selection and reserve the official splits as a clean measurement. The gap between the two is also informative in its own right, and Section 4.3 shows it detecting a data defect.

3.3 Uncertainty and significance

An accuracy computed on 500 questions is an estimate, not a constant, so every number in this report carries an interval. For `avg_accuracy` we use a percentile bootstrap over questions [2], which makes no binomial assumption and therefore also covers the $n > 1$ case where a question's score is a mean over samples. For pass@k, which is strictly 0/1 per question, we use the Wilson score interval [11]; it stays inside $[0, 1]$ and remains well behaved near the extremes, which matters here because several cells sit at or near zero.

Comparisons against the baseline use a *paired* bootstrap on the difference, since both models answer the same questions in the same order; pairing removes question difficulty as a nuisance factor and is markedly more sensitive than checking whether two intervals overlap. For binary pass@k comparisons an exact McNemar test [5] is also available. All estimators live in `src/molecularIQ_grpo/stats.py` and are unit-tested against closed-form values.

3.4 Control baseline

Because constraint generation admits many valid answers, a model can score without reasoning by emitting one small molecule every time. To measure that directly we add a *constant-answer control*: a pseudo-model that answers every question with the fixed string `{"smiles": "CC=O"}`, requiring no model and no GPU. It establishes the score reachable without reading the question at all, and any result that does not clear it is not evidence of task ability. We additionally report answer-diversity statistics – the number of distinct answers and the share taken by the most common one – for every evaluation.

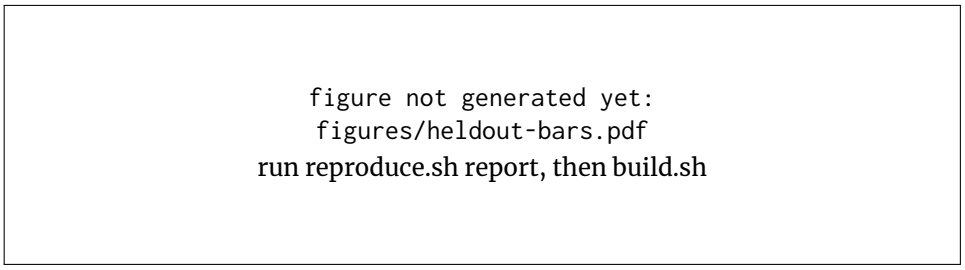

Figure 1: Accuracy on the held-out generated validation sets (greedy, pass@1). Hatched gray is the constant-answer control, solid gray the untrained baseline, colours the single-task GRPO models. Error bars are 95% bootstrap confidence intervals over questions.

4. Results

4.1 Specialization

On its own task, single-task GRPO works. On the held-out sets (Figure 1), counting rises from 0.12 at baseline to 0.25, and indexing from 0.00 to 0.25. The indexing result is the cleanest in the study: the untrained model solves essentially none of these questions, so the entire post-training score is attributable to the intervention. Both diagonal gains are significant under the paired bootstrap ($p < 0.05$).

The official splits (Figure 2) confirm the direction for two of the three families. Constraint generation improves from 0.06 to 0.14 and indexing from 0.04 to 0.08, in both cases with the task-matched model best in its column. Counting is the exception and is discussed below.

4.2 Transfer

Transfer is the negative result of this study, and it is consistent. In Table 1, the off-diagonal cells cluster at or below the untrained baseline: the count-trained model scores 0.04 on official indexing (baseline 0.04) and 0.12 on official constraint generation, and the constraint-trained model drops to 0.00 on official indexing. Nothing in the off-diagonal reaches significance as an improvement, and several cells are significant degradations.

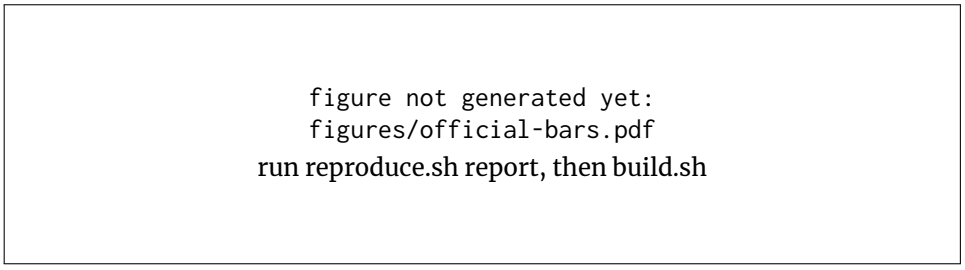

Figure 2: Accuracy on the official MolecularIQ v0.0 splits (sampled, pass@3), the headline generalization measure. Error bars are 95% Wilson intervals.

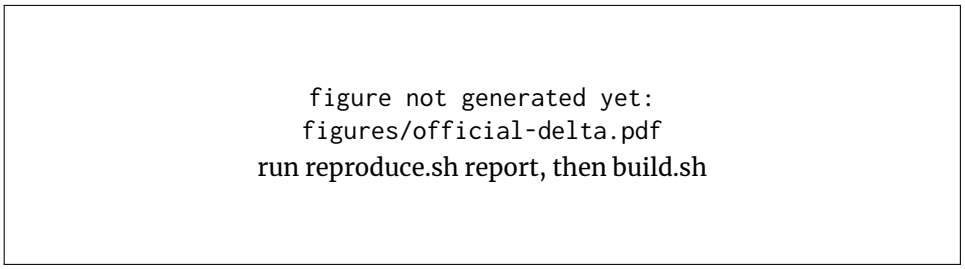

Figure 3: Change in pass@3 relative to the untrained baseline on the official splits. Rows are models, columns are tasks; outlined cells are the model's own task. Blue is improvement, red degradation.

Table 1: Full evaluation matrix. Rows are models, columns are evaluation sets. Held-out columns are greedy pass@1 (`avg_accuracy`); official columns are sampled pass@3. Bold marks the task-matched (diagonal) cell of each column. Exact values with confidence intervals and p-values are in `results/matrix/plots/*/summary.csv`.

	Held-out (pass@1)			Official (pass@3)		
Model	count	index	constr.	count	index	constr.
Constant control	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]
Qwen 0.5B untrained	0.12	0.00	0.54	0.13	0.04	0.06
GRPO count	**0.25**	0.08	0.42	**0.07**	0.04	0.12
GRPO index	0.17	**0.25**	0.48	0.14	**0.08**	0.10
GRPO constraint	0.15	0.00	**0.55**	0.07	0.00	**0.14**

The interpretation we favour is that the three families share surface form – the same prompt template, the same answer block – but not procedure. Counting requires exhaustive enumeration over a molecule graph; indexing requires enumeration *plus* positional bookkeeping; constraint generation requires constructing a molecule rather than reading one. GRPO on a single family sharpens one of these procedures and, because it also sharpens the output format that family rewards, can actively interfere with the others.

The counting column deserves separate comment because it inverts between sources: the count-trained model improves on the held-out counting set (0.12 → 0.25) but *drops* on official counting (0.13 → 0.07). This is the signature of fitting the generator rather than the task. Our generated counting questions are drawn with a capped zero-answer fraction and a bounded SMILES length, so they are narrower than the official split; a policy tuned to that narrower distribution can lose accuracy on the broader one. This is exactly the failure mode the two-source protocol was designed to expose, and it is an argument for treating the held-out sets as a diagnostic rather than as a result.

4.3 A flawed constraint set, caught by disagreement

Table 1 contains one column that does not behave like the others. On our held-out constraint set the *untrained* baseline scores 0.542 (95% CI [0.498, 0.586],

$n = 500$), and no trained model separates from it meaningfully – the constraint-trained model reaches 0.55, a difference of +0.01 that is not significant. On the official constraint split the same baseline scores 0.06. Counting and indexing agree closely across the two sources; constraint generation disagrees by roughly 9×. That pattern points at the data, not the models.

Inspecting the baseline's own answers confirms it. Across 500 questions the model produced only 59 distinct SMILES, and three molecules account for 81.4% of all answers (Table 2). The single most common answer, `C(=O)C`, was emitted for 45.8% of questions and scored 0.786 on them. The model is not solving the task; it is emitting a small molecule that happens to satisfy most constraints.

The cause is in our generator. `_build_constraint` reads a property value v from a real anchor molecule and then loosens it with a randomly chosen operator. When $v = 0$ – which is common for constructs such as bridgehead atoms, E/Z double bonds and R/S stereocenters, since most molecules have none – every operator branch yields a constraint that any molecule with zero of that property satisfies: $= 0$ holds, $\leq 0 + k$ holds, $< 0 + k$ holds, range$[0, k]$ holds, and $>$ degenerates to $= 0$ through a guard for $v < 1$. A Monte-Carlo replication of that function over 20,000 draws confirms the consequence: at $v = 0$, 100% of generated constraints are satisfied by a zero-valued molecule, and 16.5% collapse to a vacuous ≥ 0 that every valid molecule satisfies. At $v = 1$ the figure is still 49.9%.

The per-construct breakdown matches this prediction exactly. The five constructs on which the untrained baseline scores a perfect 1.00 – bridgehead, E/Z double-bond stereochemistry, unspecified double-bond stereochemistry, R/S stereocenter, and BRICS decomposition – are precisely those a small saturated molecule has none of. The single construct scoring 0.00 is molecular formula, the only one requiring an exact string match, where a lucky guess is impossible.

Critically, the zero-answer cap that prevents this failure mode for counting and indexing is not applied to constraint tasks (`data.py:405`). The same flaw is therefore present in the constraint *training* data, which explains why GRPO on constraint generation gained so little on this set: a large share of its reward was obtainable by a constant, so there was little gradient pressure toward genuine constraint satisfaction.

The constant-answer control (Section 3.4) closes the argument by measuring the trivial floor directly rather than inferring it. **[TBD: run `./scripts/reproduce.sh eval` and report the control row from Table 1; the expectation from**

Table 2: Degenerate answers from the untrained baseline on the held-out constraint set ($n = 500$, 59 distinct answers in total). Three molecules cover 81.4% of all responses.

Answer (SMILES)	Heavy atoms	Share	Reward
`C(=O)C`	3	45.8%	0.786
`C(C)C`	3	21.4%	0.495
`C(C)C(C)C`	5	14.2%	0.183
all other answers		18.6%	—

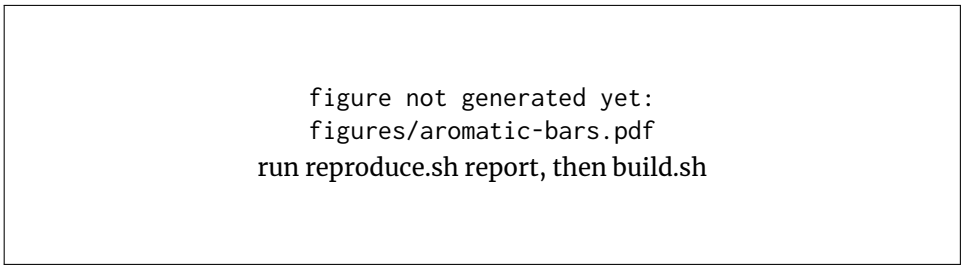

Figure 4: Single-construct study: a model trained only to count aromatic rings, against the general counting model and the baseline, evaluated both on aromatic-ring questions and on general counting.

the analysis above is a constraint-column score close to the untrained baseline's 0.54 and near zero elsewhere.]

4.4 Single-construct specialization

Narrowing the task further sharpens the same trade-off. **[TBD: fill in from `results/matrix/plots/aromatic_ring/summary.md` once the aromatic-ring model has been evaluated: the specialist's accuracy on aromatic-ring counting, and its accuracy on general counting relative to the general count model.]** The expected shape, consistent with Section 4, is a specialist that beats the general model on its own construct while losing ground on the broader counting distribution – transfer failing at construct granularity just as it fails at task granularity.

5. Limitations and Conclusion

5.1 Limitations

Several constraints on these conclusions should be stated plainly.

The constraint results are compromised on one of two sources. As Section 4.3 documents, our generated constraint questions are largely satisfiable by a trivial molecule, so the held-out constraint column measures answer size more than constraint satisfaction. We report it for transparency but draw conclusions about constraint generation only from the official split. Repairing this requires extending the zero-answer cap to constraint tasks and rejecting vacuous constraints, then regenerating the data and retraining – work we did not have the compute budget to redo here.

Single seed. Each configuration was trained once. The confidence intervals we report capture uncertainty from the finite evaluation sample, not from training stochasticity, so a small difference between two trained models should not be over-read. Repeating training across seeds is the natural next step and would let us separate run-to-run variance from real effects.

One model, one scale. All results are for a 0.5B-parameter model. Transfer between reasoning procedures may well be scale-dependent – larger models

could share more structure across tasks – so the negative transfer result should not be extrapolated upward without evidence.

Held-out sets are not independent of training data. They come from the same generator and molecule pool, which is precisely why they diverge from the official splits when the generator has a defect. They are an optimization diagnostic, not a generalization measure.

Evaluation is single-answer per question at $T = 0$ for the held-out sets. Greedy decoding understates what a model can do under sampling; the official pass@3 numbers are the fairer comparison and are the ones we report as results.

5.2 Conclusion

Single-task GRPO on MolecularIQ produces a specialist. Training on a task improves that task – most clearly for indexing, where the untrained model scores essentially zero and the trained model reaches 0.25 on held-out questions and doubles its official pass@3 – but the improvement stays local. Across the full model × task matrix, off-diagonal cells sit at or below the untrained baseline, and several are significant degradations. Counting even inverts between our held-out set and the official split, which we read as the policy fitting our generator's narrower distribution rather than the underlying task.

For the multitask work this practical leads into, that is an actionable finding. It suggests the three MolecularIQ families do not share enough procedural structure for gains to transfer implicitly, and that a naive mixture should be expected to face interference rather than synergy. Designs worth trying next are therefore ones that manage interference explicitly: balanced task sampling within a batch, per-task reward normalization so one family's reward scale cannot dominate the group advantage, and evaluation on the full matrix at every checkpoint so interference is visible while it happens rather than at the end.

The methodological lesson is at least as valuable. A benchmark number is only as trustworthy as the control it is measured against. Here, a baseline scoring 0.54 looked like a competent model until a second data source disagreed, an answer-diversity check showed three molecules covering 81% of responses, and a constant-answer control reproduced most of the score without a model at all. None of those three checks is expensive, and the evaluation harness in this repository now performs all of them by default.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*.

[2] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626.

```
./scripts/reproduce.sh datasets  # generate questions   (seed 42)
./scripts/reproduce.sh train     # 4 single-task GRPO runs
./scripts/reproduce.sh eval      # full model x eval matrix
./scripts/reproduce.sh report    # figures + summary tables
pytest -q                        # unit tests for the statistics
```

Listing 1: Full reproduction pipeline. Stages are idempotent and skip completed work.

[4] Greg Landrum. 2024. RDKit: Open-source Cheminformatics. https://www.rdkit.org. (2024).

[5] Quinn McNemar. 1947. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika*, 12, 2, 153–157.

[6] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115. (2024).

[7] Philipp Seidl, Sebastian Sanokowski, Pieter-Jan Hoedt, and Günter Klambauer. 2025. MolecularIQ: A Benchmark for Verifiable Molecular Reasoning in Large Language Models. *arXiv preprint*. Benchmark and verifier: https://github.com/ml-jku/molecullariq-core.

[8] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. In *arXiv preprint arXiv:2402.03300*.

[9] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformer Reinforcement Learning. https://github.com/huggingface/trl. (2020).

[10] David Weininger. 1988. SMILES, a Chemical Language and Information System. 1. Introduction to Methodology and Encoding Rules. *Journal of Chemical Information and Computer Sciences*, 28, 1, 31–36.

[11] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22, 158, 209–212.

Appendix A. Reproducing These Results

Every number and figure in this report is produced by the accompanying repository. From a clean checkout on a machine with the pinned environment installed (`scripts/setup_leonardo.sh`):

Randomness is pinned throughout: dataset generation and training use seed 42, evaluation uses seed 0 with greedy decoding on the held-out sets. On the pinned software stack and the same GPU model this reproduces the reported numbers exactly; across different GPU models, floating-point nondeterminism in the attention kernels can move individual accuracies by a few tenths of a percent, which is well inside the reported confidence intervals.

Figures in this report are included directly from `results/matrix/plots/`, so recompiling after a fresh run updates them automatically. Per-cell numbers, confidence intervals, p-values and answer-diversity statistics are written to `summary.csv` and `summary.md` in each group's plot directory.